

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    roc_auc_score
)

from sklearn.decomposition import PCA
```

```
In [2]: df = pd.read_csv("brca.csv")

df.head()
```

```
Out[2]:
```

	Unnamed: 0	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smoothness_mean
0	1	13.540	14.36	87.46	566.3	0.09779
1	2	13.080	15.71	85.63	520.0	0.10750
2	3	9.504	12.44	60.34	273.9	0.10240
3	4	13.030	18.42	82.61	523.8	0.08983
4	5	8.196	16.84	51.71	201.9	0.08600

5 rows × 32 columns

```
In [3]: print(df.shape)

print(df.info())

print(df.describe())
```

(569, 32)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 569 entries, 0 to 568

Data columns (total 32 columns):

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	569 non-null	int64
1	x.radius_mean	569 non-null	float64
2	x.texture_mean	569 non-null	float64
3	x.perimeter_mean	569 non-null	float64
4	x.area_mean	569 non-null	float64
5	x.smoothness_mean	569 non-null	float64
6	x.compactness_mean	569 non-null	float64
7	x.concavity_mean	569 non-null	float64
8	x.concave_pts_mean	569 non-null	float64
9	x.symmetry_mean	569 non-null	float64
10	x.fractal_dim_mean	569 non-null	float64
11	x.radius_se	569 non-null	float64
12	x.texture_se	569 non-null	float64
13	x.perimeter_se	569 non-null	float64
14	x.area_se	569 non-null	float64
15	x.smoothness_se	569 non-null	float64
16	x.compactness_se	569 non-null	float64
17	x.concavity_se	569 non-null	float64
18	x.concave_pts_se	569 non-null	float64
19	x.symmetry_se	569 non-null	float64
20	x.fractal_dim_se	569 non-null	float64
21	x.radius_worst	569 non-null	float64
22	x.texture_worst	569 non-null	float64
23	x.perimeter_worst	569 non-null	float64
24	x.area_worst	569 non-null	float64
25	x.smoothness_worst	569 non-null	float64
26	x.compactness_worst	569 non-null	float64
27	x.concavity_worst	569 non-null	float64
28	x.concave_pts_worst	569 non-null	float64
29	x.symmetry_worst	569 non-null	float64
30	x.fractal_dim_worst	569 non-null	float64
31	y	569 non-null	object

dtypes: float64(30), int64(1), object(1)

memory usage: 142.4+ KB

None

	Unnamed: 0	x.radius_mean	x.texture_mean	x.perimeter_mean	\
count	569.000000	569.000000	569.000000	569.000000	
mean	285.000000	14.127292	19.289649	91.969033	
std	164.400426	3.524049	4.301036	24.298981	
min	1.000000	6.981000	9.710000	43.790000	
25%	143.000000	11.700000	16.170000	75.170000	
50%	285.000000	13.370000	18.840000	86.240000	
75%	427.000000	15.780000	21.800000	104.100000	
max	569.000000	28.110000	39.280000	188.500000	

	x.area_mean	x.smoothness_mean	x.compactness_mean	x.concavity_mean	\
count	569.000000	569.000000	569.000000	569.000000	
mean	654.889104	0.096360	0.104341	0.088799	
std	351.914129	0.014064	0.052813	0.079720	
min	143.500000	0.052630	0.019380	0.000000	
25%	420.300000	0.086370	0.064920	0.029560	
50%	551.100000	0.095870	0.092630	0.061540	
75%	782.700000	0.105300	0.130400	0.130700	
max	2501.000000	0.163400	0.345400	0.426800	

	x.concave_pts_mean	x.symmetry_mean	...	x.radius_worst	\
count	569.000000	569.000000	...	569.000000	
mean	0.048919	0.181162	...	16.269190	

std	0.038803	0.027414	...	4.833242
min	0.000000	0.106000	...	7.930000
25%	0.020310	0.161900	...	13.010000
50%	0.033500	0.179200	...	14.970000
75%	0.074000	0.195700	...	18.790000
max	0.201200	0.304000	...	36.040000

	x.texture_worst	x.perimeter_worst	x.area_worst	x.smoothness_worst	\
count	569.000000	569.000000	569.000000	569.000000	
mean	25.677223	107.261213	880.583128	0.132369	
std	6.146258	33.602542	569.356993	0.022832	
min	12.020000	50.410000	185.200000	0.071170	
25%	21.080000	84.110000	515.300000	0.116600	
50%	25.410000	97.660000	686.500000	0.131300	
75%	29.720000	125.400000	1084.000000	0.146000	
max	49.540000	251.200000	4254.000000	0.222600	

	x.compactness_worst	x.concavity_worst	x.concave_pts_worst	\
count	569.000000	569.000000	569.000000	
mean	0.254265	0.272188	0.114606	
std	0.157336	0.208624	0.065732	
min	0.027290	0.000000	0.000000	
25%	0.147200	0.114500	0.064930	
50%	0.211900	0.226700	0.099930	
75%	0.339100	0.382900	0.161400	
max	1.058000	1.252000	0.291000	

	x.symmetry_worst	x.fractal_dim_worst
count	569.000000	569.000000
mean	0.290076	0.083946
std	0.061867	0.018061
min	0.156500	0.055040
25%	0.250400	0.071460
50%	0.282200	0.080040
75%	0.317900	0.092080
max	0.663800	0.207500

[8 rows x 31 columns]

In [4]: `df.isnull().sum()`

```
Out[4]: Unnamed: 0      0
        x.radius_mean  0
        x.texture_mean 0
        x.perimeter_mean 0
        x.area_mean    0
        x.smoothness_mean 0
        x.compactness_mean 0
        x.concavity_mean 0
        x.concave_pts_mean 0
        x.symmetry_mean 0
        x.fractal_dim_mean 0
        x.radius_se     0
        x.texture_se    0
        x.perimeter_se  0
        x.area_se       0
        x.smoothness_se 0
        x.compactness_se 0
        x.concavity_se  0
        x.concave_pts_se 0
        x.symmetry_se   0
        x.fractal_dim_se 0
        x.radius_worst  0
        x.texture_worst 0
        x.perimeter_worst 0
        x.area_worst    0
        x.smoothness_worst 0
        x.compactness_worst 0
        x.concavity_worst 0
        x.concave_pts_worst 0
        x.symmetry_worst 0
        x.fractal_dim_worst 0
        y               0
        dtype: int64
```

```
In [5]: df.duplicated().sum()
```

```
Out[5]: 0
```

```
In [6]: X = df.drop('y', axis=1)
        y = df['y']
        encoder = LabelEncoder()

        y = encoder.fit_transform(y)

        np.unique(y)
```

```
Out[6]: array([0, 1])
```

The dataset is complete since every column has 569 non-null values. Most features are continuous numerical variables, making them suitable for distance-based algorithms like KNN. Since the target is categorical, it must be encoded before model training.

```
In [7]: X = X.iloc[:,1:]
        X.head()
```

Out[7]:

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smoothness_mean	x.compactness
0	13.540	14.36	87.46	566.3	0.09779	
1	13.080	15.71	85.63	520.0	0.10750	
2	9.504	12.44	60.34	273.9	0.10240	
3	13.030	18.42	82.61	523.8	0.08983	
4	8.196	16.84	51.71	201.9	0.08600	

5 rows × 30 columns

Summary statistics such as mean, standard deviation, minimum and maximum help understand the distribution of every feature. The features have very different ranges (for example, area is in hundreds while smoothness is around 0.1). This confirms that feature scaling is necessary before applying KNN.

```
In [8]: scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(
    X_scaled,
    columns=X.columns
)

X_scaled.head()
```

Out[8]:

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smoothness_mean	x.compactness
0	-0.166799	-1.147162	-0.185728	-0.251957	0.101747	
1	-0.297446	-0.833008	-0.261106	-0.383638	0.792763	
2	-1.313080	-1.593959	-1.302806	-1.083572	0.429819	
3	-0.311646	-0.202373	-0.385500	-0.372831	-0.464730	
4	-1.684571	-0.570050	-1.658278	-1.288347	-0.737294	

5 rows × 30 columns

```
In [9]: def evaluate_split(test_size):

    X_train,X_test,y_train,y_test = train_test_split(
        X_scaled,
        y,
        test_size=test_size,
        random_state=42,
        stratify=y
    )

    model = KNeighborsClassifier(n_neighbors=5)

    model.fit(X_train,y_train)

    prediction=model.predict(X_test)
```

```

    acc=accuracy_score(
        y_test,
        prediction
    )

    return acc

```

```

In [10]: splits=[0.2,0.3,0.1]

results=[]

for s in splits:

    acc=evaluate_split(s)

    results.append(acc)

pd.DataFrame(
    {
        "Split":["80-20","70-30","90-10"],
        "Accuracy":results
    }
)

```

```

Out[10]:
   Split  Accuracy
0  80-20    0.956140
1  70-30    0.964912
2  90-10    0.964912

```

The model performs consistently across different train-test splits. Accuracy varies only slightly, indicating that KNN generalizes well and is not highly sensitive to the choice of split. The 80-20 split is selected because it provides a good balance between training and testing data.

```

In [11]: X_train,X_test,y_train,y_test=train_test_split(
        X_scaled,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )

n=len(X_train)

heuristic_k=int(np.sqrt(n))

print("Training samples:",n)

print("Heuristic K:",heuristic_k)

```

```

Training samples: 455
Heuristic K: 21

```

```
In [12]: k_values=range(
            heuristic_k-5,
            heuristic_k+6
        )

accuracy=[]

for k in k_values:

    knn=KNeighborsClassifier(
        n_neighbors=k
    )

    knn.fit(
        X_train,
        y_train
    )

    pred=knn.predict(X_test)

    accuracy.append(
        accuracy_score(
            y_test,
            pred
        )
    )
```

```
In [13]: plt.figure(figsize=(8,5))

plt.plot(
    list(k_values),
    accuracy,
    marker='o'
)

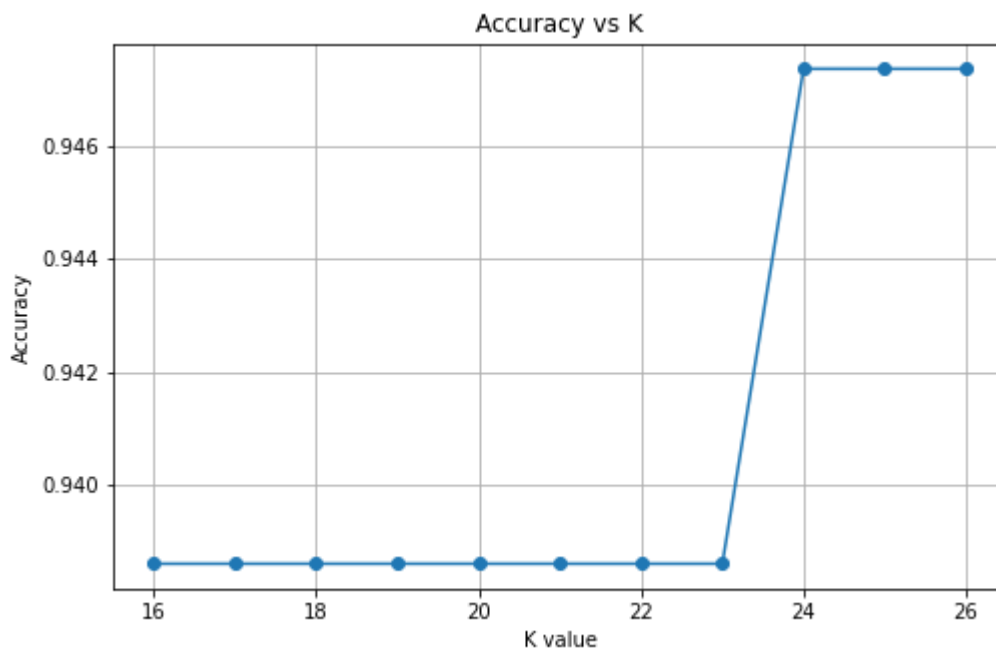
plt.xlabel("K value")

plt.ylabel("Accuracy")

plt.title("Accuracy vs K")

plt.grid()

plt.show()
```



This graph shows how model accuracy changes with different K values. Small K values can overfit because the model becomes sensitive to noise. Large K values can underfit because decision boundaries become overly smooth. The graph helps identify the K value that produces the highest testing accuracy.

```
In [14]: best_k = list(k_values)[
          np.argmax(accuracy)
        ]

        print(
            "Best K:",
            best_k
        )
```

Best K: 24

Among the tested values, K = 24 produced the highest accuracy on the test data. This value provides the best balance between overfitting and underfitting for the chosen split.

```
In [15]: pca=PCA(n_components=2)

X_pca=pca.fit_transform(X_scaled)

X_pca=pd.DataFrame(
    X_pca,
    columns=['PC1', 'PC2']
)
```

PCA reduces the original 30-dimensional dataset into only two principal components. The reduction is performed only for visualization. The original features are still used for final model training.

```
In [16]: def plot_boundary(k):

          model=KNeighborsClassifier(
              n_neighbors=k
          )
```



```

model.fit(
    X_pca,
    y
)

x_min,x_max=X_pca.iloc[:,0].min()-1,X_pca.iloc[:,0].max()+1
y_min,y_max=X_pca.iloc[:,1].min()-1,X_pca.iloc[:,1].max()+1

xx,yy=np.meshgrid(
    np.arange(x_min,x_max,0.05),
    np.arange(y_min,y_max,0.05)
)

Z=model.predict(
    np.c_[xx.ravel(),yy.ravel()]
)

Z=Z.reshape(xx.shape)

plt.figure(figsize=(6,5))

plt.contourf(
    xx,
    yy,
    Z,
    alpha=0.3
)

plt.scatter(
    X_pca.iloc[:,0],
    X_pca.iloc[:,1],
    c=y,
    edgecolor='black'
)

plt.title(
    f"KNN Decision Boundary K={k}"
)

plt.xlabel("PC1")
plt.ylabel("PC2")

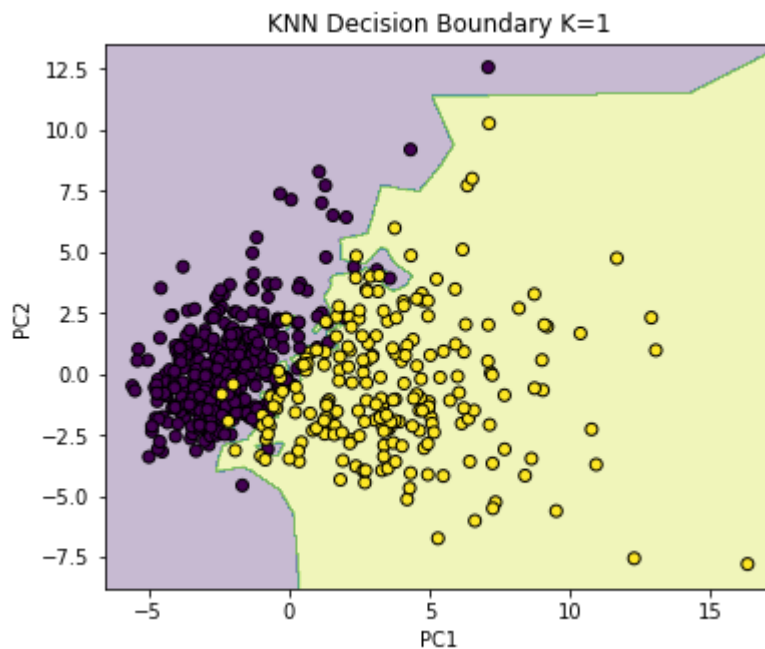
plt.show()

```

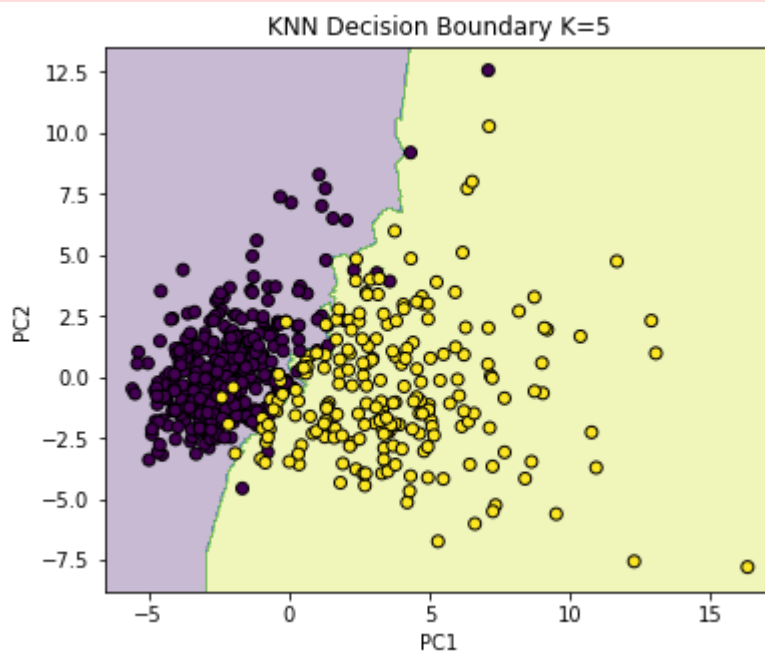
In [17]: **for** k **in** [1,5,10,20]:

```
plot_boundary(k)
```

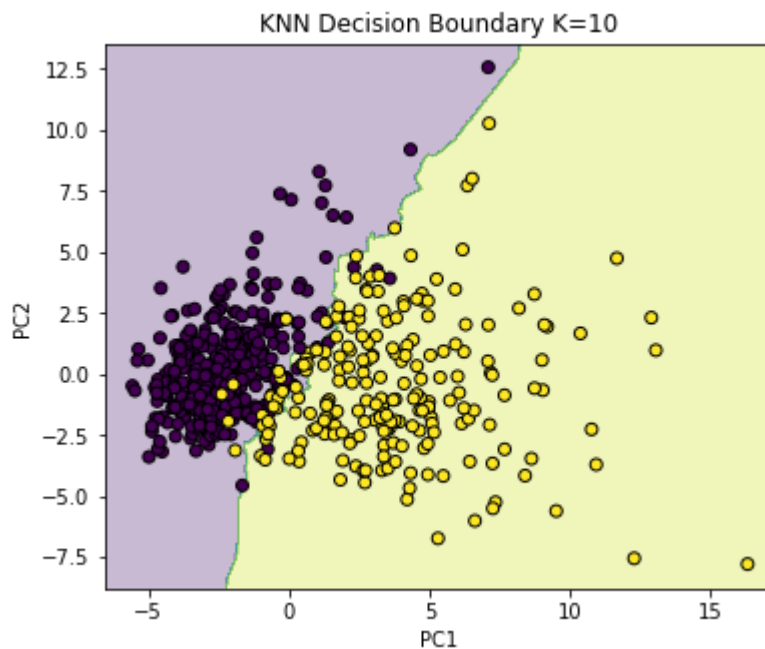
c:\Users\shree\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have valid feature names, but KNeighborsClassifier was fitted with feature names
warnings.warn(



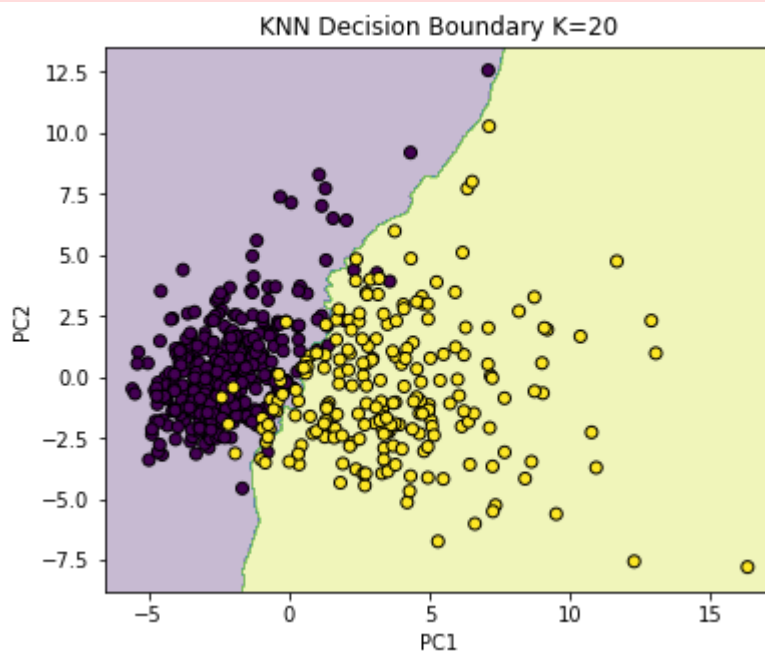
```
c:\Users\shree\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have valid feature names, but KNeighborsClassifier was fitted with feature names
warnings.warn(
```



```
c:\Users\shree\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have valid feature names, but KNeighborsClassifier was fitted with feature names
warnings.warn(
```



c:\Users\shree\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have valid feature names, but KNeighborsClassifier was fitted with feature names
warnings.warn(



K = 1 Interpretation Decision boundary is highly irregular. Model memorizes the training data. Very low bias but very high variance. Highly prone to overfitting. K = 5 Interpretation Boundary becomes smoother. Better balance between bias and variance. More stable predictions than K = 1. K = 10 Interpretation Decision boundary becomes even smoother. Noise influence decreases. Model begins to generalize better. K = 20 Interpretation Decision boundary is very smooth. Model becomes less sensitive to individual observations. If K becomes too large, important local patterns may be ignored, leading to underfitting.

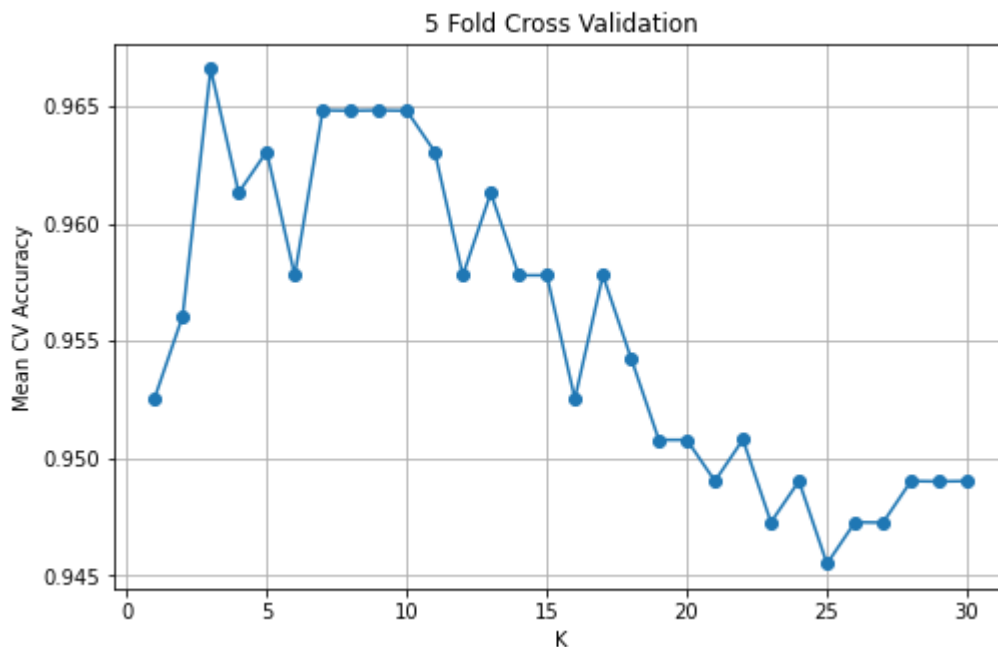
```
In [18]: k_values=range(1,31)

cv_results=[]

cv=KFold(
    n_splits=5,
```

```
    shuffle=True,  
    random_state=42  
)  
  
for k in k_values:  
  
    model=KNeighborsClassifier(  
        n_neighbors=k  
    )  
  
    scores=cross_val_score(  
        model,  
        X_scaled,  
        y,  
        cv=cv,  
        scoring='accuracy'  
    )  
  
    cv_results.append(  
        scores.mean()  
    )
```

```
In [19]: plt.figure(figsize=(8,5))  
  
plt.plot(  
    list(k_values),  
    cv_results,  
    marker='o'  
)  
  
plt.xlabel("K")  
plt.ylabel("Mean CV Accuracy")  
plt.title("5 Fold Cross Validation")  
  
plt.grid()  
plt.show()
```



The graph shows average validation accuracy for different K values. Cross-validation evaluates performance across multiple train-test combinations. It provides a more reliable estimate than a single train-test split. The highest point indicates the most stable K value.

```
In [20]: best_cv_k = list(k_values)[
            np.argmax(cv_results)
        ]

        print(
            "Best CV K:",
            best_cv_k
        )
```

Best CV K: 3

Although K = 24 performed best on one train-test split, K = 3 achieved the highest average accuracy across all folds. This suggests K = 3 generalizes better to unseen data. Therefore, K = 3 is selected as the final model.

```
In [21]: final_model=KNeighborsClassifier(
            n_neighbors=best_cv_k
        )

        final_model.fit(
            X_train,
            y_train
        )

        y_pred=final_model.predict(
            X_test
        )
```

```
In [23]: print("accuracy: ", accuracy_score(
            y_test,
            y_pred
        ))
        print("precision: ", precision_score(
```

```

    y_test,
    y_pred
))
print("recall: ", recall_score(
    y_test,
    y_pred
))
print("F1 score: ", f1_score(
    y_test,
    y_pred
))

```

```

accuracy: 0.9385964912280702
precision: 0.972972972972973
recall: 0.8571428571428571
F1 score: 0.9113924050632912

```

Accuracy (93.86%) Nearly 94% of all tumor samples were classified correctly. Indicates strong overall model performance. Precision (97.30%) When the model predicts cancer, it is correct almost every time. Very few false positives are produced. Recall (85.71%) The model correctly detects about 86% of actual cancer cases. Around 14% of malignant cases are still missed. In medical diagnosis, improving recall is extremely important. F1 Score (91.14%) Indicates an excellent balance between precision and recall. Shows the classifier performs consistently across both metrics.

Although the model achieved 93.86% accuracy, the Recall was only 85.71%.

Insight: This means that while the overall classification performance is high, approximately 14% of actual positive cancer cases could still be missed, making Recall an essential metric for healthcare applications.

```

In [24]: print(
    classification_report(
        y_test,
        y_pred
    )
)

```

	precision	recall	f1-score	support
0	0.92	0.99	0.95	72
1	0.97	0.86	0.91	42
accuracy			0.94	114
macro avg	0.95	0.92	0.93	114
weighted avg	0.94	0.94	0.94	114

```

In [25]: cm=confusion_matrix(
    y_test,
    y_pred
)

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

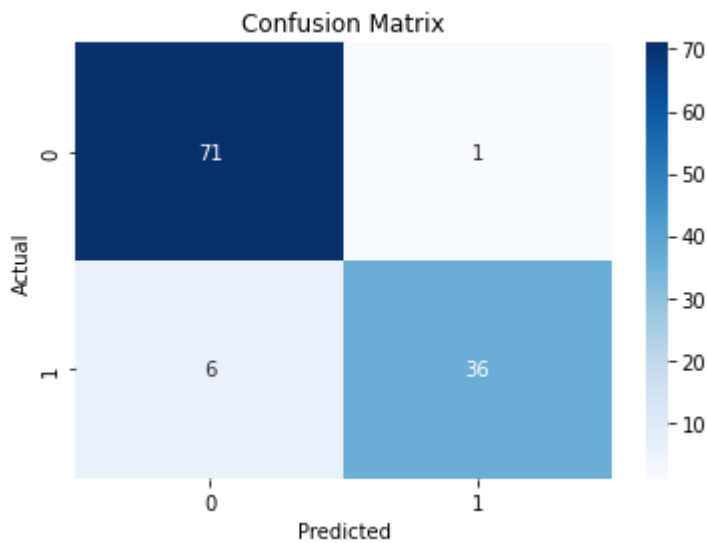
```

```
plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()
```



```
In [27]: y_prob=final_model.predict_proba(
          X_test
        )[:,1]
auc=roc_auc_score(
      y_test,
      y_prob
    )

print(
      "AUC Score:",
      auc
    )
```

AUC Score: 0.982473544973545

```
In [28]: fpr,tpr,thresholds=roc_curve(
          y_test,
          y_prob
        )

plt.figure(figsize=(7,5))

plt.plot(
    fpr,
    tpr,
    label=f"AUC={auc:.3f}"
)

plt.plot(
    [0,1],
    [0,1],
    linestyle='--'
)
```

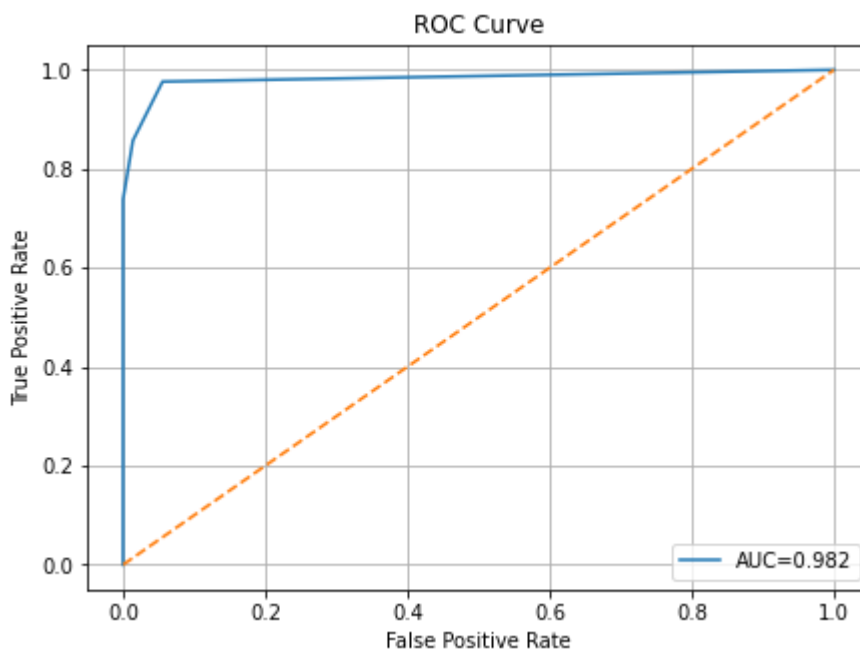
```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.legend()

plt.grid()

plt.show()
```



COMPARATIVE ANALYSIS

Regression (Lab 3)	Classification (Lab 4)
MAE	Accuracy
MSE	Precision
RMSE	Recall
R ² Score	F1 Score
Measures numerical prediction error	Measures correctness of predicted class

Regression predicts continuous values (e.g., house price), whereas classification predicts discrete categories (e.g., benign or malignant cancer).

Regression (Error-based) Measures how far the predicted value is from the actual value. Even a small prediction difference contributes to the error. Lower error indicates a better model.

Examples:

MAE MSE RMSE Classification (Decision-based) Measures whether the predicted class is correct or incorrect. Focuses on correct decisions instead of numerical differences. Higher metric values indicate better performance.

Examples:

Accuracy Precision Recall F1 Score ROC-AUC

R² Score vs Accuracy | R² Score | Accuracy | | ----- | -----
 ----- | | Used in regression | Used in classification | |
 Measures how much variance is | Measures percentage of correctly classified samples |
 explained by the model | Range: (-∞ to 1) | Range: 0 to 1 | | Higher is better | Higher is better
 |

Lab 3 Multiple Regression: R² = 0.8063 Lab 4 KNN: Accuracy = 93.86%

R² shows how well continuous values are explained, while Accuracy measures how many class labels are predicted correctly.

RMSE	F1 Score
Regression	Classification
Measures average prediction error magnitude	Balances Precision and Recall
Lower is better	Higher is better

Lab 3 RMSE = 10685.06 Lab 4 F1 Score = 0.9114

RMSE measures numerical prediction error, whereas F1 Score measures the balance between correctly identifying positive cases and minimizing false alarms.

MAE	Confusion Matrix
Measures average absolute prediction error	Shows detailed classification outcomes
Gives one numerical error value	Displays TP, TN, FP, FN
Lower is better	Used to compute Accuracy, Precision, Recall and F1

MAE tells how much predictions differ from actual values, whereas the Confusion Matrix explains where the classifier makes correct and incorrect decisions.

Continuous Prediction (Regression) Predicts numerical values. Output can take any value within a range. Performance is evaluated using prediction errors.

A) Predicting house prices from area, bedrooms and other features -

Evaluation Metrics:

MAE MSE RMSE R² Score Classification Tasks Predicts predefined categories. Output belongs to a fixed class. Performance depends on correct classification.

B) Predicting whether a breast tumor is benign or malignant -

Evaluation Metrics:

Accuracy Precision Recall F1 Score ROC-AUC Confusion Matrix

Regression and classification use different evaluation metrics because they solve different problems. Regression metrics such as MAE, MSE, RMSE, and R^2 measure how close predicted numerical values are to the actual values and how well the model explains the data.

Classification metrics evaluate the correctness of predicted class labels. Accuracy measures overall correctness, while Precision, Recall, F1 Score, and the Confusion Matrix provide deeper insight into prediction performance.

In medical diagnosis, Accuracy alone is not enough because a model may miss patients with the disease (false negatives). Recall is more important as it measures how well the model identifies actual positive cases. ROC-AUC is also valuable because it evaluates the model's ability to distinguish between classes across different thresholds.

Overall, regression evaluation focuses on minimizing prediction errors, while classification evaluation emphasizes correct classification, especially reducing false negatives in critical applications such as cancer diagnosis.

Lab 3 focused on minimizing numerical prediction errors using MAE, MSE, RMSE, and R^2 Score. Lab 4 focused on maximizing correct classification using Accuracy, Precision, Recall, F1 Score, and ROC-AUC. The regression model achieved $R^2 = 0.8063$, explaining about 81% of the variation in house prices. The classification model achieved 93.86% accuracy and 0.9825 ROC-AUC, indicating excellent classification capability.

1. Why is KNN called lazy learning?

Because KNN does not build a model during training. It stores training data and performs calculations only when prediction is required.

2. Why scaling is required?

KNN uses distance calculations. Features with larger numerical ranges dominate smaller features.

3. $\sqrt{n}$ heuristic K selection

A common rule chooses root of n , where n is training samples. It provides a starting point for tuning.

4. Why cross validation is better?

Because it evaluates the model on multiple data subsets and reduces dependency on one random split.

5. Effect of K on bias variance

Small K:

Low bias High variance Sensitive to noise

Large K:

High bias Low variance Smoother boundaries

6. Why recall matters in cancer detection?

Missing a cancer case (false negative) is dangerous. High recall reduces missed malignant cases.

7. Limitation of large K

Large K may classify everything according to the majority class and underfit the data.